Autoformalizing Fulton–Harris in Arend

A constructive variant integrated into arend-lib

Revised 12 October 2026; Maschke interface updated 20 October 2026
source audit: 8 October 2026

Integration target. Incorporate a constructive variant directly into arend-lib. First establish a reproducible compiler/library baseline, then complete a small source-aligned pilot in Lectures 1–2. Extend the existing representation and linear-algebra modules; add character theory and Lie theory in their mathematical namespaces. Use no excluded middle or choice axioms. Replace classical existence arguments with constructions or explicitly recorded constructive reformulations; unresolved replacements remain research gaps. Exercises needed for the theory receive formal statements with intentional proof holes; dependent results retain explicit proof-status records.

Destination and first deliverable

The target is Fulton and Harris, *Representation Theory: A First Course*, with a constructive coverage map for all 26 lectures and Appendices A–F, using `~/Dropbox/AlgGeom/Fulton-Harris.pdf`. Exercises important for the theory receive formal statements and named proof holes; other exercises are deferred unless selected. Record whether each target preserves the original statement, reformulates it, or restricts its inputs. A constructive counterpart does not certify the unrestricted classical statement.

The first milestone covers finite-dimensional splitting and averaging, constructive Schur variants, character identities, orthogonality for explicitly constructed split irreducibles, and an exact S_3 example. A smaller pilot precedes it: the 12 source items on page 7. This tests retrieval, statement translation, proof repair, and semantic review before expensive Lie-theoretic work begins.

What the 8 October source audit established

- The checked-out library has 447 `.ard` source files, including representations, intertwiners, subrepresentations, averaging, Schur's lemma, tensors, matrices, and trace. Existing hypotheses need a constructivity audit.
- The representation subtree is small: nine files, approximately 450 lines. `Characters.ard` is 19 lines and proves only the value at the identity beyond its definition and a helper. File names considerably overstate readiness for Lectures 1–2.
- Complex numbers and substantial fundamental-theorem-of-algebra machinery exist. The interface between constructive complex numbers and the discrete-field linear-algebra API requires deliberate work.
- No dedicated Lie-algebra, root-system, highest-weight, Young-tableau, or smooth-manifold development was located in the source inventory and targeted searches. These are substantial planned developments.

Validation limit. The available 1.12 executable failed on shared `\Cat` syntax, then raised an exception. This audit does not certify arend-lib. M0 must resolve the compiler pairing; see page 15.

1 Reusable foundations and their actual limits

Paths in the tables are relative to `arend-lib/src/`. "Present" means that a definition or proof body was inspected; it does not override the typechecking limitation on page 1. "Gap" means not found in the audited interfaces and searches, and should be rechecked with the symbol index at M0.

Area	Present in the checkout	Remaining work for the book
Representations	`LinRepres`, `TrivialAction`; actions linear over a ring.	Bundle finite dimension and distinguish abstract representations from chosen matrix presentations.
Maps and subobjects	`IntertwiningMap`, `KernelLRepres`, `ImageLRepres`; representation category and module of intertwiners.	Finite-dimensional intertwiners, invariant vectors, quotients and isomorphism-facing convenience lemmas.
Direct sums	`ProductLRepres` with inclusions, projections and a coproduct map.	Finite indexed sums, basis/dimension formulas and decomposition interfaces. Binary products model finite direct sums.
Schur	`SchurLemma`: an intertwiner between library-irreducibles is zero or an equivalence.	Replace the strong `IsIrreducible` definition, which implies excluded middle given a nonzero vector; adapt Schur and its callers.
Maschke	`SumOverGroup`, `mean_func`, `mean-func-preserve`.	Construct linear retractions from finite basis/independence data, then average and exhibit invariant complements.
Characters	`Character E bv g`, `Character.of-1`; character depends explicitly on a basis witness.	Basis independence, class functions, sums, tensors, duals, orthogonality, completeness and character tables.
Linear algebra	`FinModule`, dimension uniqueness, surjection splitting; matrix/linear-map conversion; Smith-based kernel/image finiteness.	Based/split subspaces and explicit rank computations over suitable discrete fields; separate treatment over constructive $\mathbb{C}$.
Trace and spectra	`Trace-prod`, `Trace-conjugation`, characteristic polynomial, Cayley–Hamilton.	Coordinate-free trace; trace of a projector; root-to-eigenvector bridge. `eigen-root` proves the opposite direction.
Multilinear algebra	`TensorProductModule`, `tensorLift` and uniqueness; bilinear maps; associative algebras.	Tensor bases, dual/Hom representations, tensor powers, symmetric/exterior powers, contractions and Schur functors.
Finite groups	Finite sets/sums, symmetric groups, cycles and signs; monoid rings.	Group-algebra representation equivalence, restriction/induction, Frobenius reciprocity, Young combinatorics and relevant finite-field examples.

Exact source anchors. The representation definitions are under `Algebra/Group/Representation.ard` and `Algebra/Group/Representation/`. The intertwiner record is `IntertwiningMap`, renamed from the misspelled `InterwiningMap`. Linear algebra is in `Algebra/Module/`, `Algebra/Linear/VectorSpace.ard`, and `Algebra/Linear/Matrix/`. Supporting constructions include `Algebra/Group/Symmetric.ard`, `Algebra/Ring/MonoidRing.ard`, and `Algebra/Algebra.ard`. Preserve these names in retrieval and adapters.

2 Constructive foundations and statement design

1. Choice-free constructive policy

Use Arend's type theory and justified constructive library constructions. Exclude `Logic.Classical` (`alem`, `achoice`) and `Logic.Choice` (`acchoice`, `adchoice`), including transitive uses. Do not replace them with new axioms or equivalent blanket assumptions. This plan chooses a choice-free constructive variant; finite enumeration and proved finite-choice lemmas remain available. Audit theorem types and definitions as well as proof bodies and meta-generated terms.

2. Separate exact scalar computations from constructive complex numbers

Use commutative rings for action/trace identities and an explicit inverse of $|G|$ for averaging. Use discrete fields only where their decisions are constructively supplied, initially $\mathbb{Q}$ for concrete examples. Construct exact splitting fields, such as the needed cyclotomic extensions, with verified operations and factorization data; their APIs are work to do.

`Algebra/Linear/VectorSpace.ard:43` requires `DiscreteField`; `Arith/Complex.ard:69` supplies `ComplexField : Field`. Do not add decidable equality to arbitrary $\mathbb{C}$. Prove scalar-extension bridges and identify precisely which results transfer. A theorem over an exact field alone is not yet the book's theorem over constructive $\mathbb{C}$. Distinguish equality, negated equality and positive invertibility/apartness evidence whenever an algorithm divides or tests a pivot. The explanation on page 5 makes these distinctions concrete. Prioritize direct complex constructions and generalize library interfaces that unnecessarily require `DiscreteField`; exact-field examples supplement this work.

3. Replace the strong irreducibility premise

`IsIrreducible` quantifies over *all* predicate subrepresentations. It includes the zero representation; adding nonzeroness is insufficient. Indeed, for any proposition P, the invariant submodule

$$W_P = \{x \mid P \vee x = 0\}$$

is allowed by `SubLRepres`. Given $v \neq 0$, the assertion that W_P is zero or the whole space yields $\neg P$ or P. Thus this source definition, with a known nonzero vector, implies excluded middle. This is a mathematical consequence of the inspected definitions, not a newly checked Arend theorem. Page 4 gives the argument step by step using the one-dimensional trivial representation over $\mathbb{Q}$.

Replace arend-lib's existing `IsIrreducible` definition with a reviewed constructive formulation. M1 must settle its generality, nontriviality and nonzero-element evidence, prove basic one-dimensional examples constructively, and migrate Schur's lemma and its callers. The old premise must not remain the default irreducibility interface.

For the finite-dimensional pilot, investigate a provisional notion testing invariant embeddings $j : K^k \hookrightarrow V$ over a field K, with a chosen basis and a linear left inverse $rj = I$ constructed from suitable finite data; r need not be equivariant. For finite-dimensional V of positive dimension, require each such embedding to have $k = 0$ or be an isomorphism. This is not the general replacement for arbitrary modules. Validate its closure properties; for Schur's kernel/image argument, prove admissibility using exact linear algebra where applicable. Over $\mathbb{C}$, prove scalar commutants directly for constructed representations rather than assume a zero-or-invertible test for every scalar.

4. Keep finite data and truncated existence distinct

`FinModule.isFinModule` is truncated existence of a finite basis. Use chosen bases for computations, then prove invariance or unique existence before eliminating truncation into a scalar or dimension. Follow `FinModule.dimension-pair`; do not choose bases globally. Supply eigenpairs/factorizations as data only for explicitly restricted lemmas, and construct that data in concrete applications.

3 Why the irreducibility premise is too strong

The current definition asks too much constructively: even proving the one-dimensional trivial representation irreducible would imply excluded middle. The issue comes from the combination of an unrestricted notion of submodule and the explicit assertion that each submodule is zero or everything.

The definition. What counts as a submodule

In `Algebra/Group/Representation/Irreducible.ard`, `IsIrreducible` says every invariant submodule is either zero or the whole space. This looks standard. But `SubLRepres` allows membership defined by *any proposition*; a basis or a decision procedure for membership is not required. Over $\mathbb{Q}$, these modules are vector spaces.

The example. Encode an arbitrary proposition in a subspace

Take $V = \mathbb{Q}$ with the trivial group action. Let P be any proposition whose truth we do not know, and define

$$W_P = \{x \in \mathbb{Q} \mid x = 0 \text{ or } P\}.$$

This is a valid invariant subspace without deciding P. Zero belongs to it. For addition, if either membership proof supplies P, the sum belongs; otherwise both summands are zero. Scalar multiplication also preserves membership, and the trivial action preserves every element.

If P holds, every rational number belongs to W_P, so $W_P = \mathbb{Q}$. If $\neg P$ holds, only zero belongs, so $W_P = \{0\}$. These are conditional statements; neither decides P.

The consequence. Strong irreducibility would decide P

Suppose we have proved irreducibility using the existing definition. Applied to W_P, it gives $W_P = \{0\}$ or $W_P = \mathbb{Q}$. Either outcome settles P:

1. **If $W_P = \{0\}$, then $\neg P$.** Otherwise P would put 1 in W_P, contradicting $1 \neq 0$.
2. **If $W_P = \mathbb{Q}$, then P.** Since $1 \in W_P$, its membership means $1 = 0$ or P. The first possibility is impossible.

Thus we obtain $P \vee \neg P$ for an arbitrary proposition P: the law of excluded middle.

The proposed replacement. A finite-dimensional pilot only

For the finite-dimensional pilot, investigate irreducibility relative to explicitly based, linearly split invariant submodules. A splitting is a linear retraction onto the submodule that fixes its elements. This is a provisional notion for that setting, not a universal definition of irreducibility. We cannot generally supply the required data for W_P without already settling P.

A finite basis restricts the submodules W being tested, not logically the ambient module M. Applied to an infinite-dimensional M, the test may miss important infinite-dimensional invariant submodules. Over a general ring, even a submodule of a finite free module need not be free. Keep the general module API open to infinite-dimensional objects, including Verma modules.

The proposed notion still needs mathematical validation. For the kernel/image proof of Schur's lemma, prove that the relevant kernels and images belong to the permitted class.

Replacement in arend-lib. The existing Schur proof is valid conditional on its old premise. Replace `IsIrreducible` with the reviewed constructive definition and adapt Schur's lemma and all callers, recording changed hypotheses or conclusions. This is a planned semantic change, not merely a new predicate alongside the old default. The argument above is mathematical explanation, not a newly typechecked Arend result.

4 What the scalar-field distinction means

Some existing linear-algebra code assumes it can always decide whether a number is exactly zero. Constructive complex numbers do not provide that general zero test. This is a restriction on particular algorithms and interfaces, not a prohibition on doing complex linear algebra.

Zero tests. Why rational and complex numbers differ

For a rational number, test whether its numerator is zero. For an arbitrary constructive complex number, increasingly accurate approximations need not settle exact equality: even knowing $|z| < 10^{-100}$ does not establish $z = 0$. The number might be zero or merely very small.

These complex numbers are exact mathematical objects, not floating-point approximations. Equality can be proved in particular cases. What is unavailable is an assumed procedure that decides equality for every pair of complex numbers.

The library interfaces. What the extra assumption supplies

In `Algebra/Field.ard`, `Field` supports constructive field operations and inversion with suitable evidence. `DiscreteField` additionally provides a zero-or-invertible decision for every element, from which equality is decidable. The current `VectorSpace` requires this stronger interface. Complex vector spaces can instead use `LModule` over `ComplexField`; appropriate definitions and theorems should be generalized to that setting.

Evidence for division. Three different assertions

Assertion	What it supplies
$z = 0$	A proof of exact equality.
$z \neq 0$	A proof that assuming $z = 0$ leads to a contradiction.
$z \# 0$ (apart from zero)	Positive evidence of separation from zero, for example a rational $\varepsilon > 0$ with $\lvert z\rvert \geq \varepsilon$, sufficient to construct an inverse.

Constructively, negated equality does not automatically give this positive evidence. In the library's `Field`, apartness from zero is expressed using `Inv z`. A Gaussian-elimination step that divides by a pivot must have the required evidence; it cannot silently assume an arbitrary zero test.

Scalar extension. Allowing more scalars

Take matrices with rational entries and let them act on $\mathbb{C}^n$ instead of $\mathbb{Q}^n$. Matrix identities transfer, but other properties need separate proofs. For example,

$$A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad A^4 = I.$$

This gives a representation of the cyclic group of order four over both fields. It has no invariant rational line: such a line would give a rational eigenvalue λ with $\lambda^2 = -1$. Over $\mathbb{C}$, it splits into eigenlines with eigenvalues i and $-i$. Thus irreducibility does not automatically transfer, even though the matrix identity does.

Consequence for this project. Develop complex representation theory directly where possible, generalize unnecessarily restrictive interfaces, and require explicit evidence where a construction needs it. Rational examples are a useful starting point, not a replacement for the complex theory.

5 Constructive replacements for classical proof steps

Every source item gets both an original statement and a reviewed constructive target. Mark their relation as `same`, `reformulated`, or `restricted`; use `research-gap` if no suitable target is settled. A restriction is visible scope, not a proof of the unrestricted statement.

Book argument	Constructive deliverable	Remaining obligation
Obtain basis or splitting data for an arbitrary invariant submodule	Construct an ordinary complement from finite data, then average the projection using $\lvert G\rvert^{-1}$.	Separate finite basis extension from obtaining bases for unrestricted predicate submodules; see below.
Repeatedly choose an irreducible summand	Build finite decomposition algorithms for exact, presented inputs; verify every projection and intertwiner.	Prove termination and completeness for that input class. Finite dimension alone does not decide whether a module is reducible.
Apply scalar Schur using an eigenvalue	Compute the commutant of constructed split irreducibles; prove it consists of scalar matrices.	A general eigenpair-relative lemma has explicit hypotheses. Prove required eigenpair data in each application.
Identify multiplicities and classify	Compare supplied finite decompositions using verified Hom spaces. Construct a finite family and prove exhaustiveness in the specified category.	Prove splitting-field and base-change results; never assume the desired classification as an input certificate.
Character inner products	Use $\lvert G\rvert^{-1}\sum_g \chi_V(g^{-1})\chi_W(g)$ over an exact splitting field.	Prove the complex conjugate version separately. Trace of the invariant projector gives dimension only with justified finite-rank data.
Choose weights and a Cartan subalgebra	Begin with explicit split Cartan/root data and finite weight decompositions; construct the classical series.	General existence and conjugacy require constructive proofs or an explicit reformulation.
Use compactness, integration or global geometry	Specify effective local charts, estimates, integration data and the exact conclusion needed.	Audit each analytic theorem; retain unresolved constructive replacements as research gaps.

Finite-dimensional complements. Over a discrete field, supplied finite bases for $W \subseteq V$ allow finite elimination or basis extension to construct a retraction; no choice axiom is needed. Over constructive $\mathbb{C}$, use positive independence evidence, such as an invertible maximal minor of the inclusion matrix. Average the resulting retraction when G is finite and $\lvert G\rvert$ is invertible. The obstacle is obtaining the finite data for an arbitrary `SubLRepres` from ambient finite dimension alone, as W_P illustrates. Treat complement construction from those data as a reusable lemma.

The root-selection boundary. `Arith/Complex/FTA.ard` supplies a spectrum in the completion of finite multisets. Use this constructive output where appropriate; it is not an individual-root selector. The inspected single-root proof in `Arith/Complex/FTA/Classical.ard:155` calls `acchoice` and is outside this plan. Retrieve declarations rather than stale `FTA.Choice` comments. Richman's choice-free development of the fundamental theorem of algebra is the primary reference for the reformulated spectrum.

A concrete first route. Work out S_3 over $\mathbb{Q}$ by exact matrices. Verify decomposition maps, scalar commutants and character pairings, then extend the constructions to $\mathbb{C}$ without introducing complex equality tests. General finite-group theory follows after explicit splitting-field and group-algebra decomposition infrastructure. If a stronger complex statement requires new analysis, register that bridge rather than importing choice.

6 The first source-aligned pilot

Use Lectures 1–2 to exercise three kinds of task: a reuse wrapper, a short new consequence, and a theorem requiring new infrastructure. The following 12 items are an initial backlog of constructive targets. Locations identify the source inspiration; the table does not assert equivalence to every original statement. Record each relation using page 6.

ID	Book location	Deliverable and proof approach	Dependencies
P01	§1.1, pp. 3–5	Replace `IsIrreducible`; define based finite representations and pilot subobjects; relate actions to $G \to \mathrm{GL}(V)$.	Constructive statement review; existing actions
P02	§1.1; Ex. 1.2	Define direct sum, dual and Hom actions. State invariant Hom = intertwiners with a named exercise hole.	P01; dual/Hom basis facts
P03	Schur 1.7(1), p. 7	Reuse the kernel/image proof pattern with constructive admissibility evidence; package the inverse.	P01; exact kernel/image bases
P04	Prop. 1.5, p. 6	Construct a retraction from finite data, then average to obtain an invariant complement.	P01; basis/pivot data; $\lvert G\rvert^{-1}$
P05	Cor. 1.6, p. 6	Compute a decomposition for the presented pilot inputs; verify maps and simplicity of summands.	P04; constructive splitting procedure
P06	Schur 1.7(2), p. 7	Prove scalar commutants for the constructed split irreducibles by finite matrix equations.	P01; explicit matrices
P07	Prop. 1.8, p. 7	Uniqueness of multiplicities in supplied decompositions with verified Hom dimensions.	P02, P05, P06; off-diagonal Hom vanishing
P08	§2.1, p. 13	Basis-independent character, class invariance, and $\chi_V(1) = \dim V$.	P01; change of basis and trace
P09	Prop. 2.1, p. 13	First establish sum, tensor and dual character formulas; leave the exterior-square conjunct explicitly pending.	P02, P08; tensor bases
P10	Ex. 2.5, p. 14	State the permutation-character fixed-point formula; leave its proof as a named exercise hole.	P08; `PermRepr`; finite sums
P11	Prop. 2.8, p. 15	Average onto invariants; relate trace to dimension when the image has certified finite-rank data.	P08; idempotent trace; exact rank
P12	Thm. 2.12, p. 16	Orthogonality for the constructed split irreducibles, using the inverse-character pairing.	P02, P06, P09, P11; Hom vanishing

Order. Begin P01, P04 and P08; register P02/P10 exercise holes. Validate the admissibility API with P03 and the S_3 constructions before generalizing P05–P07. P09 remains partial until the exterior-square formula is included. Prove that the inverse-character pairing specializes to the book's conjugate pairing when making that comparison.

A concrete regression example. Construct the trivial, sign and two-dimensional standard representations of S_3 over $\mathbb{Q}$. Prove that the permutation representation on three letters splits as $1 \oplus V_{\mathrm{std}}$ and derive the character rows $(1, 1, 1)$, $(1, -1, 1)$ and $(2, 0, -1)$ on the identity, transposition and three-cycle classes. Verify the class sizes $1, 3, 2$ and orthogonality by exact arithmetic. Then prove base change to $\mathbb{C}$. Exhaustiveness requires its own constructive completeness proof.

Pilot exit. Review every statement. Check non-exercise proofs where dependencies permit; retain P02's exercise component and P10 as holes. Report downstream results that depend on these holes as conditional or pending, including any affected S_3 checks. Exercise holes are not proof-search tasks.

7 Theory exercises remain explicit holes

An exercise is important for the theory when later definitions or arguments use its result, or when it supplies a construction needed by the chosen proof route. Record that role and the consuming declarations. Formalize its full statement in the appropriate arend-lib mathematical module, with explicit hypotheses and an intentionally unfinished proof body. Review its constructive formulation before registering the hole. The autoformalizer must leave that body open rather than solve the exercise.

Representation. Use an Arend goal expression such as `{?FH_L01_Ex1_2}` as the proof body. Give it a stable source identifier, cite the exercise in the declaration's doc comment, and list it in the coverage manifest. The hole is part of the planned deliverable, not a missing task to be retried. Do not replace it with an axiom, insert a solution under a helper name, or silently discharge it by applying an existing theorem. An exercise hole must not conceal excluded middle, choice, or an unreviewed classical existence principle. Record any reformulation in its source entry.

Immediate examples. Exercise 1.2 identifies the invariant vectors in $\mathrm{Hom}(V, W)$ with intertwiners. P02 builds the representation constructions, but leaves this identification as an exercise hole. Exercise 2.5 gives the fixed-point formula for permutation characters; P10 is a statement and hole, not a request to prove that formula. These results are used in the character theory, so their downstream dependencies remain visible.

```
id: FH.L01.Ex1.2
kind: exercise
theory_role: "Identifies invariant Hom with intertwiners"
formal_target: Algebra.Group.Representation.Hom:invariantHom
status: intentional-exercise-hole
statement_review: required
source_relation: same  # confirm during constructive statement review
proof_policy: leave-hole
goal_name: FH_L01_Ex1_2
consumers: [FH.L02.Thm2.12]
```

The declaration name is proposed; the source identifier remains stable even if the mathematical API changes. The consumers list is populated from the actual proof route, including transitive dependencies in generated reports.

Downstream development. Continue writing later theory against the reviewed exercise statement where Arend permits it, while recording the dependency on its hole. Such a result is not an unconditional checked proof. If the checker cannot elaborate a consumer of an open declaration, keep it pending, or check a clearly conditional helper with the exercise statement as an explicit parameter. Neither approach closes the original exercise hole.

Validation. Check signatures and all available proof bodies. Match intentional exercise goals to the manifest by declaration, location and goal name. Distinguish their expected diagnostics from unexpected type errors, unregistered holes and remaining downstream obligations. A run containing expected goals is a development report, not a clean proof certification. The subset reported as fully verified must have no holes in its dependency closure. M0 must establish how the pinned checker reports goals and consumers.

Completion policy. A theory milestone may be complete with the registered exercise statements left open. Report proved assertions, intentional exercise holes, conditional/pending consumers and deferred items separately. Non-exercise gaps still require proofs or explicit pending status. Leave exercises outside the theoretical dependency graph deferred unless selected; their solutions are not part of the automatic proof backlog.

8 Whole-book coverage map

Fulton–Harris is organized pedagogically. Implementation should follow dependencies, especially the appendices used by later lectures. This table accounts for the full destination. All targets below mean constructive variants subject to page 6; general existence and analytic results require statement-level feasibility review.

Book unit	Target	Development needed
Lectures 1–2	Finite representations and characters	Pilot; explicit splitting fields and group-algebra matrix blocks; constructive completeness, multiplicities and isotypic projectors.
Lecture 3	Induction, group algebras, real representations	Finite coset data, induction, Frobenius reciprocity, $K[G]$-module equivalence, exterior powers and certified real/complex descent.
Lectures 4–6	Symmetric/alternating groups, finite GL_2, Weyl's construction	Partitions, tableaux, Young symmetrizers, symmetric functions, Frobenius character formula and Schur functors; finite-field and alternating-group branches.
Lectures 7–8	Lie groups, Lie algebras, exponential	Algebraic brackets first; constructively specified manifolds, tangent spaces and exponential maps form a separate research track.
Lectures 9–10	Structure and low dimensions	Explicit ideals, nilpotence/solvability witnesses; constructive Engel/Lie variants and radicals. Audit eigenvector and subspace choices.
Lectures 11–13	$\mathfrak{sl}_2$, $\mathfrak{sl}_3$, plethysm	Explicit raising/lowering operators and polynomial modules; classification initially for supplied finite weight decompositions.
Lecture 14	Semisimple structure	Killing form, explicit split Cartan data, roots/weights and highest weights; general existence is a separate obligation.
Lectures 15–19	Classical series	$\mathfrak{sl}_n$, symplectic and orthogonal Lie algebras; root data, dominant weights and tensor constructions, including low-rank coincidences.
Lecture 20	Spin	Clifford algebras, spin modules and triality; Lie-group realization depends on the analytic track.
Lectures 21–22	Classification and exceptional types	Finite root data, Dynkin algorithms, explicit reconstruction and exceptional modules. State which supplied structures are classified.
Lectures 23–25	Groups and character formulas	Weight lattices, representation rings, Weyl/Freudenthal/Kostant formulas, branching; homogeneous spaces and Bruhat geometry are explicit additional obligations.
Lecture 26	Real forms and representations	Explicit conjugations and real/quaternionic structures; constructive compact-group theory remains a research dependency. Track both character-formula routes.
Appendices A–B	Symmetric functions; multilinear algebra	Move tensors, duals, exterior/symmetric powers and symmetric-function identities before their first use in Lectures 2–6.
Appendices C–D	Semisimplicity; Cartan subalgebras	Constructive Cartan criterion, splitting, Jordan data and Weyl groups; audit Cartan existence/conjugacy before admitting statements.
Appendices E–F	Levi/Ado; invariant theory	Explicit enveloping-algebra constructions; constructive Levi/Ado variants, invariants and Capelli identity. Register unresolved replacements.

Coverage discipline. Report source relation and proof status on separate axes. A proved restricted variant, an open generalization and a reviewed exercise hole are three different deliverables.

9 How to build the missing Lie theory

Algebraic interfaces. Small reusable structures

Define a Lie algebra over a commutative ring as a module with a bilinear, alternating bracket and Jacobi identity. Define homomorphisms, ideals, subalgebras, quotients, the adjoint action and Lie modules. Prove that commutators in an associative algebra give a Lie algebra. Build $\mathfrak{gl}(V)$ and $\mathfrak{sl}(V)$ using endomorphisms and trace, with matrix models for calculations. Keep the zero bracket and characteristic-sensitive cases in view.

First independent milestone. $\mathfrak{sl}_2$ and explicit representations

Prove the relations

$$[h, e] = 2e, \qquad [h, f] = -2f, \qquad [e, f] = h.$$

Construct $\mathrm{Sym}^n(K^2)$ with explicit bases and coefficient formulas. Prove simplicity for the selected admissible-submodule notion and specify the inverses of positive integers used in characteristic zero. Construct Clebsch–Gordan maps and their inverses. First classify modules carrying an explicit finite weight decomposition and suitable highest-weight data; do not silently assume every module comes with these witnesses. Proving their existence for a broader class is a separate task. This milestone validates the Lie APIs before general classification.

Structural results. Move prerequisites before users

Audit Engel/Lie, Cartan's criterion, Jordan decomposition and semisimple splitting for uses of eigenvectors, maximal subspaces and nonconstructive case splits. Start with explicit split Cartan subalgebras and root-space data, then seek constructive existence and conjugacy results corresponding to Appendices C–D. Define finite root systems independently and implement Dynkin classification with checked certificates. Prove the bridge to Lie algebras: classifying root data alone does not classify all complex Lie algebras.

For general highest weights, plan explicit universal-enveloping-algebra, PBW and highest-weight-module infrastructure if that proof route is selected. Keep the Lie-module interface unrestricted in dimension so it can accommodate Verma modules; the finite-dimensional pilot's submodule criterion is local to that pilot. Use explicit ordered monomials and reductions. After the $\mathfrak{sl}_2$ pilot, audit existence proofs for choices of maximal submodules. If an alternative construction follows the book more directly, record that proof route and its dependencies instead. No temporary "highest-weight theorem" axiom counts toward completed results.

Character formulas. A proof, not only a formula evaluator

Represent formal characters in the group ring of a weight lattice, reusing monoid-ring machinery where appropriate. Develop Weyl-group actions, positive roots, the denominator identity and the representation-theoretic argument needed for the Weyl character formula. Separate a verified combinatorial algorithm for weights from the theorem identifying its output with the character of a module. Then derive dimension, multiplicity, tensor product and restriction formulas and test low ranks against explicit modules.

Lie groups and geometry. A constructive research track

The library contains topology, analysis and a synthetic derivative interface, but no ready-made manifold/Lie-group package was found. Audit the real and complex differentiable setting before choosing an implementation. Specify the data and estimates needed for inverse-function/exponential results, coverings, integration of Lie-algebra maps and compact-group averaging. Finite-group averaging does not supply Haar integration. Treat global existence theorems as research gaps until a constructive formulation and proof route are reviewed. Develop homogeneous/flag spaces and Bruhat arguments under explicit hypotheses. Record any algebraic replacement and its relation to the source; omitted geometry remains visible in coverage.

10 The autoformalization workflow

Automate one dependency-ready assertion at a time. The controller must separate source interpretation, declaration design, proof generation and acceptance. First classify theory exercises as intentional holes (page 8). A typecheck verifies a formal statement, not its fidelity to the book.

1. **Extract and align.** Use the PDF text layer for search and page images for formulas. The text extraction visibly confuses S_d, A_d, $\mathfrak{sl}_n$, $\oplus$, superscripts and theorem numbers (e.g. 2.8). Store both printed page and physical PDF page; validate each mapping rather than assume one constant offset throughout the scan.
2. **Normalize the claim.** Create a short mathematical statement, explicit hypotheses, notation bindings and a proposed Arend declaration. Split multi-part propositions. Resolve phrases such as "canonical", "unique" and "by Schur" into separate obligations. Review the statement before broad proof search. Review constructivity: record a same-statement translation, a reformulation or an input restriction. Specify the data replacing each classical choice. Unsettled replacements are research gaps.
3. **Retrieve exact dependencies.** Search definitions by name, signature shape, class hierarchy and usages. Supply the generator with actual declarations, hypotheses, import paths and nearby proof patterns. Filter by the constructive dependency policy; classical lemmas and the strong `IsIrreducible` premise are not default proof templates.
4. **Plan a proof and register gaps.** Prefer an application of an existing theorem, then a short composition of lemmas, then new foundation work. Register an exercise dependency as a named intentional hole and track its consumers. For other missing lemmas, create proof tasks and postpone consumers. Never insert a result as an untracked assumption.
5. **Generate, elaborate, repair.** Work in an isolated working copy of the target arend-lib module. Try local proof steps using `rewrite`, `exts`, `equation`, `linarith`, `cases` and `simplify` metas where their domains apply. Feed exact diagnostics and relevant goals back into a bounded repair loop; save unsuccessful attempts. Exclude intentional exercise holes from generation and repair.
6. **Review and certify.** Check source fidelity, quantifiers, nonzeroness, characteristic, finiteness, equality versus isomorphism and all additional hypotheses and constructive source relation. Inspect the elaborated result, then perform a fresh typecheck with secondary checking. A fully verified result has no holes in its dependency closure. Report intentional exercise goals and their consumers separately; reject unregistered gaps, forbidden axioms and hidden classical strength in hypotheses. A parameterized classical principle is still outside the constructive target.
7. **Integrate a small accepted increment.** Add the proof to its arend-lib mathematical module, with provenance and regression examples. Check affected consumers and library tests. Add a reusable proof pattern to retrieval only after review. Integrate registered exercise holes as statements, preserving their status and downstream dependencies in coverage reports.

Use the tools already in this checkout. The CLI exposes `-ss` for names, `-ps` for signature-shape retrieval, `-fu` for usages, `-ch` for hierarchy and `-sc` for scope. The retrieval commands support `-json`; `-p` prints elaborated definitions after checking. In particular, `-ps` is retrieval of existing declarations, not a general automatic proof synthesizer. Verify these capabilities against the newly pinned executable before relying on them.

Stopping rule. Start with a modest fixed per-assertion retry/token budget, for example five repair cycles. Classify exhaustion as source ambiguity, missing library lemma, elaboration failure or proof-search failure. Do not repeat essentially identical attempts or silently weaken the theorem. Revisit the task once the missing dependency or interface issue is resolved.

11 Integration into arend-lib and evidence records

Use arend-lib's existing `arend.yaml`, source tree and test suite. Definitions and theorems belong in mathematical namespaces; book numbers belong in documentation and provenance records. The layout below proposes extensions to arend-lib. This planning task updates only the TeX and PDF in the repository's `docs/` directory.

```
arend-lib/
  arend.yaml                          # existing library configuration
  src/
    Algebra/Group/Representation/     # extend Schur, Maschke, characters
    Algebra/Module/                   # duals, tensors, exterior/symmetric powers
    Algebra/Linear/                   # finite bases, rank, split subspaces
    Algebra/Group/Symmetric/          # representation theory of symmetric groups
    Combinatorics/                    # partitions and tableaux
    Algebra/Lie.ard, Algebra/Lie/     # new Lie algebra/module/root/weight APIs
    Topology/, Analysis/, AG/         # group/geometry foundations as needed
    Arith/Complex/                    # constructive spectra and base change
  test/Algebra/                      # exact examples and API regressions
  docs/fulton-harris/                 # source manifest, dependencies, coverage
  tools/autoformalization/            # retrieval, generation/checking controller
```

A source record. Use a machine-readable schema such as JSON Lines. For the finite-dimensional constructive Maschke variant, record at least:

```
id: FH.L01.Prop1.5
kind: proposition
source: {printed_page: 6, pdf_page: 21, label: "Proposition 1.5"}
original: "Every invariant subspace has an invariant complement."
constructive: "Build and average a retraction for a based invariant subspace."
source_relation: restricted
data: [finite_group, inverse_of_group_order, based_inclusion]
linear_algebra: discrete_field_or_certified_pivots
formal_target: Algebra.Group.Representation.MaschkeLemma:finiteDimComplement
dependencies: [GAP.finiteDimRetraction, mean-func-preserve]
status: planned
statement_review: pending
proof_check: not_run
axiom_policy: no_excluded_middle_or_choice
axiom_audit: pending
```

The target name above is a design placeholder, not an existing theorem. Once checked, add the exact compiler/library/source hashes, declaration signature hash, import/declaration dependencies, command, exit status, log location, proof route, axiom dependencies and reviewer outcome.

States. Use `extracted`, `statement-reviewed`, `blocked-on-dependency`, `candidate`, `checked`, and `accepted`; permit an explicit `deferred` state. "Checked" and "accepted" are separate. Add `intentional-exercise-hole` and `conditional-on-exercises`; neither means fully proved. Reused results map to library declarations after statement review. Keep exercise holes open.

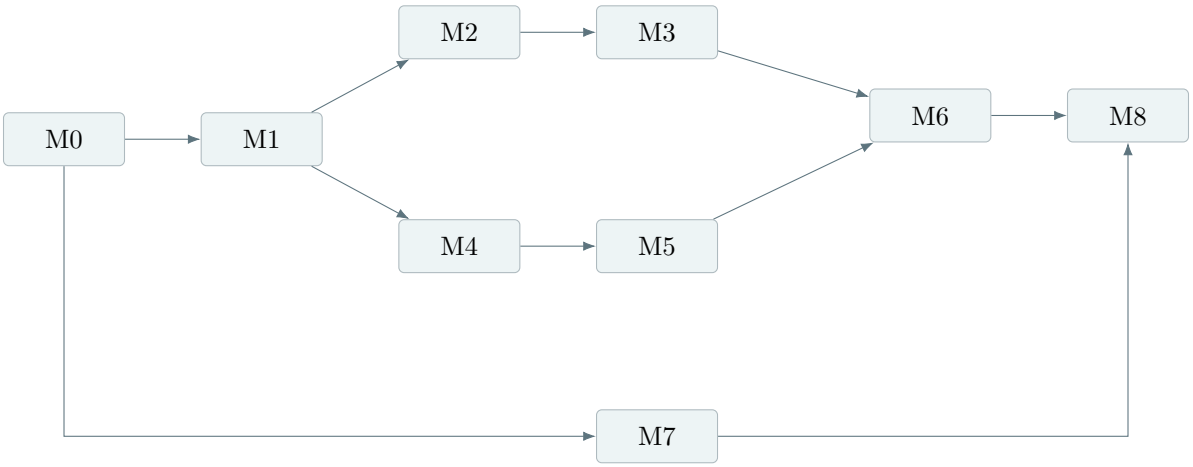
Dependency graph. Maintain both the source-level mathematical DAG and the actual declaration/import graph, including types and definitions. Use them for work order, rechecking and axiom provenance. Track a missing lemma as a named node with intended signature, consumers and acceptance condition. If elaborated dependency export is insufficient, start with a conservative import audit and label its limitation rather than claim exact axiom usage.

Library maintenance. Replace `IsIrreducible` and migrate its theorems and callers in a checked change, documenting semantic differences. Extend mathematical modules directly in arend-lib with affected-consumer checks and tests under `arend-lib/test/`. Use mathematical names and source provenance. Exclude classical dependencies from accepted results.

12 Milestones, dependencies and stopping gates

These are acceptance milestones, not elapsed-time promises. Later stages have more uncertainty than the finite-group pilot. Each stage incorporates its mathematics and tests into arend-lib under the constructive policy. Registered exercise holes satisfy the statement deliverable; their consumers remain conditional/pending. Research gaps prevent claims of full theory completion.

Stage	Deliverable	Exit evidence	Needs
M0	Reproducible baseline	Pin compiler/extensions; check required modules; establish goal and axiom reporting; reject an invalid-proof control.	None
M1	Constructive interfaces	Replace `IsIrreducible` and migrate callers; validate constructive examples and pilot interfaces; check P01/P03/P04/P08 and register P02/P10 holes.	M0
M2	Finite-group core	Exact S_3, pilot and Prop. 2.1 variants; construct splitting fields and completeness for specified inputs. Verify bridges to $\mathbb{C}$; retain exercise holes.	M1; linear and multilinear bridges
M3	Remaining Part I	Constructive Lectures 3–6 and A–B: induction, Young theory, Schur functors and finite-field examples; record source relations.	M2
M4	Algebraic Lie pilot	Lie APIs; explicit $\mathfrak{sl}_2$ modules and Clebsch–Gordan; classification for reviewed weight-decomposed inputs.	M1; relevant B material
M5	Split Lie structure	Split Cartan/root data, highest weights; constructive Engel/Lie and C–D variants. General existence gaps tracked separately.	M4; C–D; M3 tensor tools
M6	Advanced algebra	Explicit classical/exceptional series, spin, Dynkin reconstruction and character formulas; reviewed E–F variants.	M3, M5
M7	Groups and geometry	Constructive analytic foundations and reviewed group/geometry variants. Unresolved replacements remain research gaps.	M0; analytic audit; M4–M6 as needed
M8	Coverage reconciliation	Every source item mapped to its constructive target or gap; fresh proof/axiom audit; holes and consumers reported. Full completion requires all agreed gaps closed.	M2–M7



The diagram shows major work streams; the table retains cross-dependencies. M4 need not wait for all symmetric-group and finite-field material. M7 can begin its foundation audit early and joins the final completion gate.

13 Quality, effort measurement and project risks

Acceptance. Distinguish proved results from exercise holes

1. **Semantic review:** the mathematical assertion and formal signature agree in quantifiers, hypotheses and exceptional cases. Record the original and constructive statements and their relation. A reviewed restriction is a valid variant; the stronger source claim remains separate. Do not add hypotheses merely to make proof search succeed.
2. **Proof review:** check the declaration and dependency closure from source with no goals, placeholders, excluded middle or choice axioms. Confirm the expected modules were processed. Exercise holes receive statement review; their consumers remain conditional/pending.

Test rejection of an invalid proof of $0 = 1$ in $\mathbb{N}$ and detection of a choice-dependent declaration. Record the trusted compiler, prelude and extensions. These are harness checks, not consistency tests.

Regression cases. Check zero/trivial representations, basis changes, S_3, and characteristic $p \mid |G|$, where Maschke's inverse is unavailable. Later add $\mathfrak{sl}_2$ weights and low-rank characters. Check that simplicity avoids arbitrary proposition-submodule dichotomies and complex base change requires no equality oracle.

Measurement. Estimate after the pilot

Calibrate over roughly ten working days: M0, M1 statements/proofs and one substantive bridge. If M0 consumes this timebox, reschedule the proof pilot. Report source relations, proof statuses, exercise holes, blocked dependencies, reuse, attempts, human review time, generation cost and checking time. Separate wrappers, routine algebra, bridges and major theorems.

Estimate remaining work per task class using observed median and upper-quartile effort, with separate uncertainty ranges for untested Lie and geometry foundations. This audit supports no automation-success rate or completion date. Budget exercise statements, excluding solutions to their intentional holes. Helper-lemma counts are not book-theorem counts.

Risk	Early signal	Response
Compiler/library mismatch	Parse failures before the target proof	Rebuild and pin at M0; record source and executable hashes.
Wrong formal statement	Proof succeeds only after adding assumptions	Freeze reviewed signatures; return to dependency planning.
Field/basis mismatch	Implicit-instance failures or illegal extraction of a basis	Use explicit adapters and basis-independence lemmas.
Hidden classical strength	A lemma needs arbitrary zero tests or unrestricted subobject dichotomies	Audit hypotheses, not just axiom names; revise the target explicitly.
Hidden analytic dependency	A Lie-algebra argument invokes compact groups or geometry	Route to M7, or supply a proved algebraic replacement and record it.
Proof-search stagnation	Repeated failures with the same missing fact	Stop retries; create a reusable lemma task.
Overstated coverage	Exercise holes or their consumers counted as proved	Report proof status, intentional holes and deferred items separately.

14 Reproducibility and source notes

Audit provenance. The constructive revision is dated 12 October 2026. Relevant irreducibility, submodule and FTA definitions were reread for this revision. Inventory counts and the compiler experiment below are from 8 October; the compiler experiment was not rerun. The audited Git revision was `17fe0520676cefeb38a9fcb3568f94c0ef9028fb` (30 September 2026, "Fix serialization"). `arend-lib/` is tracked in this same repository, not a separate submodule. Its `arend.yaml` declares language/library version 1.12, `src`, `test`, `bin`, and the extension entry point `org.arend.lib.StdExtension`. Existing untracked `docs/` material was present before this planning task.

Book identity. The supplied PDF has 559 physical pages and a searchable text layer. SHA-256: `83a1d9cb639aa8c548de87682dc4f4a799dfde06c8f46977a386a524c1e3eeb1`. The contents pages and selected early theorem pages were visually inspected. The study read the early representation/character statements and audited relevant library bodies; it did not verify every proof in all 26 lectures. The later roadmap is a dependency assessment from the book's structure and the source inventory, not a completed theorem-by-theorem extraction.

Baseline experiment. Java reported OpenJDK 21.0.10. The available `cli-1.12.0-full.jar`, dated 19 September 2026, had SHA-256 `ba31ff5c574b0d79b76163b62c5fc871493027f87db452c0cd9c913a2fa1be1f`. From the repository root, this command exited with status 1:

```
java -Xmx4g -jar cli/build/libs/cli-1.12.0-full.jar arend-lib \
  Algebra.Group.Representation.Irreducible \
  Algebra.Group.Representation.MaschkeLemma \
  Algebra.Group.Representation.Characters -r -c
```

It failed while processing shared dependencies: the first reported error was `Equiv:15:24`, rejecting `\Cat` ; further parser errors preceded `IndexOutOfBoundsException` in name resolution. This is evidence that the available executable/source pairing is unsuitable, not evidence that Schur's lemma or Maschke's mathematics is false. No library proof was counted as freshly verified by this experiment.

Proposed M0 commands; not executed as part of the plan.

```
./gradlew :cli:jarDep :arend-lib:meta:classes
# Verify the rebuilt executable's help, version and hashes, then:
java -Xmx4g -jar cli/build/libs/cli-1.12.0-full.jar arend-lib -r -c
java -Xmx4g -jar cli/build/libs/cli-1.12.0-full.jar arend-lib -t -r -c
# During the pilot, check the affected arend-lib modules, for example:
java -Xmx4g -jar cli/build/libs/cli-1.12.0-full.jar \
  arend-lib Algebra.Group.Representation.Irreducible -r -c
```

Use a clean process and no stale binary caches for acceptance. If the rebuilt CLI can route to an existing daemon, use its documented `-no-daemon` option so fresh-check flags take effect. Do not assume options in current source documentation are implemented by an older bundled JAR.

Primary references. The local source files named throughout this document are the evidence for the audit. Operational references are `ARCHITECTURE.md`, `settings.gradle.kts`, `arend-lib/meta/build.gradle.kts`, and `cli/src/main/java/org/arend/frontend/doc/ConsoleMain-arguments.md`. Meta implementations are registered in `arend-lib/meta/src/main/java/org/arend/lib/StdExtension.java`. The official Arend documentation and upstream arend-lib repository were consulted on 8 October 2026 for orientation; availability claims here refer to the local snapshot, which may differ from upstream.